



Universidad CENFOTEC

Escuela de Ingeniería del Software

Principios de programación

Código: SOFT-01

Avance 1

Enlace al repositorio:

<https://github.com/julian26-salazar04/mision-orbita.git>

Julian Salazar Solís

Profesora:

Veronica Mora Lezcano

Sección: SCV3 – Periodo: C3-2026

Octubre, 2026

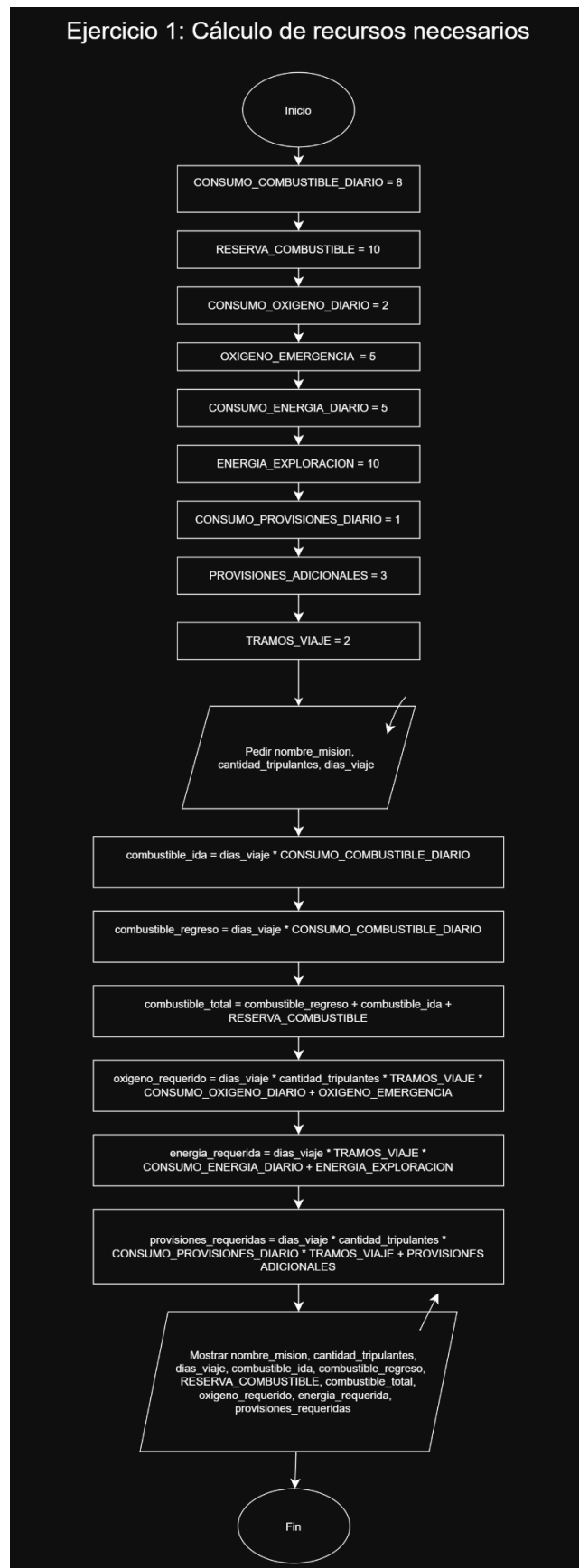
EJERCICIO 1: Recursos

Tabla de variables:

Nombre	Tipo de dato	Categoría	Descripción	Valor inicial/restricciones
CONSUMO_COMBUSTIBLE_DIARIO	int	Constante	Combustible que se consume por día de viaje	8
RESERVA_COMBUSTIBLE	int	Constante	Combustible adicional para emergencias	10
CONSUMO_OXIGENO_DIARIO	int	Constante	Oxígeno que consume cada tripulante por día	2
OXIGENO_EMERGENCIA	int	Constante	Oxígeno adicional de emergencia	5
CONSUMO_ENERGIA_DIARIO	int	Constante	Energía que consume la nave por día de viaje	5
ENERGIA_EXPLORACION	int	Constante	Energía para operaciones de exploración	10
CONSUMO_PROVISIONES_DIARIO	int	Constante	Provisiones que consume cada tripulante por día	1
PROVISIONES_ADICIONALES	int	Constante	Provisiones adicionales	3
TRAMOS_VIAJE	int	Constante	Cantidad de tramos del viaje (ida y regreso)	2
nombre_mision	str	Entrada	Nombre de la misión	Texto
cantidad_tripulantes	int	Entrada	Número de tripulantes	Entero positivo

dias_viaje	int	Entrada	Días estimados para llegar al destino	Entero positivo
combustible_ida	int	Salida	Combustible necesario para llegar	$\text{dias_viaje} * \text{CONSUMO_C OMBUSTIBLE_DIARIO}$
combustible_regreso	int	Salida	Combustible necesario para regresar	$\text{dias_viaje} * \text{CONSUMO_C OMBUSTIBLE_DIARIO}$
combustible_total	int	Salida	Combustible total con reserva	ida + regreso + reserva
oxigeno_requerido	int	Salida	Oxígeno necesario para la misión	Calculado
energia_requerida	int	Salida	Energía necesaria para la misión	Calculado
provisiones_requeridas	int	Salida	Provisiones necesarias para la misión	Calculado

Diagrama de flujo:



Código en python:

```
# =====
# Archivo: ejercicio1_recursos.py
# Curso: SOFT-01 Principios de Programación 1 - Sección SCV3
# Integrantes: Julián Salazar Solís
# Fecha: 30/09/2026      Versión: 1.1
# Descripción: Calcula los recursos necesarios para una misión.
# =====

# -----
# Declaración de constantes
# Se definen aquí para evitar números mágicos en las fórmulas.
# -----
CONSUMO_COMBUSTIBLE_DIARIO = 8
RESERVA_COMBUSTIBLE = 10
CONSUMO_OXIGENO_DIARIO = 2
OXIGENO_EMERGENCIA = 5
CONSUMO_ENERGIA_DIARIO = 5
ENERGIA_EXPLORACION = 10
CONSUMO_PROVISIONES_DIARIO = 1
PROVISIONES_ADICIONALES = 3
TRAMOS_VIAJE = 2

# -----
# Entrada de datos
# -----
nombre_mision = input("Nombre de la misión: ")
cantidad_tripulantes = int(input("Cantidad de tripulantes: "))
dias_viaje = int(input("Días estimados para llegar al destino: "))

# -----
# Procesamiento
# El regreso dura lo mismo que la ida, por eso usa los mismos días.
# La reserva se suma una sola vez al total, para emergencias.
# -----
combustible_ida = dias_viaje * CONSUMO_COMBUSTIBLE_DIARIO
combustible_regreso = dias_viaje * CONSUMO_COMBUSTIBLE_DIARIO
combustible_total = combustible_ida + combustible_regreso +
RESERVA_COMBUSTIBLE

# Oxígeno, energía y provisiones cubren ida y regreso, por eso el
consumo
```

```

# diario se multiplica por TRAMOS_VIAJE. Las cantidades fijas
(emergencia, exploración y adicionales) se suman una sola vez.
oxigeno_requerido = dias_viaje * CONSUMO_OXIGENO_DIARIO * TRAMOS_VIAJE
* cantidad_tripulantes + OXIGENO_EMERGENCIA
energia_requerida = dias_viaje * CONSUMO_ENERGIA_DIARIO * TRAMOS_VIAJE
+ ENERGIA_EXPLORACION
provisiones_requeridas = dias_viaje * cantidad_tripulantes *
CONSUMO_PROVISIONES_DIARIO * TRAMOS_VIAJE + PROVISIONES_ADICIONALES

# -----
# Salida de resultados
# -----

print()
print("Misión:", nombre_mision)
print("Tripulantes:", cantidad_tripulantes)
print("Duración estimada hasta el destino:", dias_viaje, "días")
print("Combustible para llegar:", combustible_ida, "unidades")
print("Combustible para regresar:", combustible_regreso, "unidades")
print("Reserva de combustible:", RESERVA_COMBUSTIBLE, "unidades")
print("Combustible total requerido:", combustible_total, "unidades")
print("Oxígeno requerido:", oxigeno_requerido, "unidades")
print("Energía requerida:", energia_requerida, "unidades")
print("Provisiones requeridas:", provisiones_requeridas, "unidades")

```

Casos de prueba:

ID	Ejercicio	Escenario	Datos de entrada	Resultado esperado	Resultado obtenido	Estado
CP-01	Ejercicio 1	Ejemplo de la consigna	Misión: Explorador 1 Tripulantes: 3 Días: 5	Combustible ida: 40 Combustible regreso: 40 Reserva: 10 Total: 90 Oxígeno: 65 Energía: 60 Provisiones: 33	Misión: Explorador 1 Tripulantes: 3 Duración estimada hasta el destino: 5 días Combustible para llegar: 40 unidades Combustible para regresar: 40 unidades Reserva de combustible: 10 unidades Combustible total requerido: 90 unidades Oxígeno requerido: 65 unidades Energía requerida: 60 unidades Provisiones requeridas: 33 unidades	Aprobado
CP-02	Ejercicio 1	Valores mínimos (1 tripulante, 1 día)	Misión: Mini 1 Tripulantes: 1 Días: 1	Combustible ida: 8 Combustible regreso: 8 Reserva: 10 Total: 26 Oxígeno: 9 Energía: 20 Provisiones: 5	Misión: Mini 1 Tripulantes: 1 Duración estimada hasta el destino: 1 días Combustible para llegar: 8 unidades Combustible para regresar: 8 unidades	Aprobado

					Reserva de combustible: 10 unidades Combustible total requerido: 26 unidades Oxígeno requerido: 9 unidades Energía requerida: 20 unidades Provisiones requeridas: 5 unidades	
CP-03	Ejercicio 1	Misión larga con más tripulantes	Misión: Titan Tripulantes: 4 Días: 10	Combustible ida: 80 Combustible regreso: 80 Reserva: 10 Total: 170 Oxígeno: 165 Energía: 110 Provisiones: 83	Misión: Titan Tripulantes: 4 Duración estimada hasta el destino: 10 días Combustible para llegar: 80 unidades Combustible para regresar: 80 unidades Reserva de combustible: 10 unidades Combustible total requerido: 170 unidades Oxígeno requerido: 165 unidades Energía requerida: 110 unidades Provisiones requeridas: 83 unidades	Aprobado

Evidencia de ejecución:

CP-01: Ejemplo de la consigna

```
> & C:\Users\xmlsc\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/xmlsc/OneDrive/Desktop/Principios de programacion #1/mision-orbita/avance1/ejercicio1_recursos.py"
Nombre de la misión: Explorador 1
Cantidad de tripulantes: 3
Días estimados para llegar al destino: 5

Misión: Explorador 1
Tripulantes: 3
Duración estimada hasta el destino: 5 días
Combustible para llegar: 40 unidades
Combustible para regresar: 40 unidades
Reserva de combustible: 10 unidades
Combustible total requerido: 90 unidades
Oxígeno requerido: 65 unidades
Energía requerida: 60 unidades
Provisiones requeridas: 33 unidades
PS C:\Users\xmlsc\OneDrive\Desktop\Principios de programacion #1\mision-orbita>
```

CP-02: Valores mínimos

```
PS C:\Users\xmlsc\OneDrive\Desktop\Principios de programacion #1\mision-orbita> & C:\Users\xmlsc\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/xmlsc/OneDrive/Desktop/Principios de programacion #1/mision-orbita/avance1/ejercicio1_recursos.py"
Nombre de la misión: Mini 1
Cantidad de tripulantes: 1
Días estimados para llegar al destino: 1

Misión: Mini 1
Tripulantes: 1
Duración estimada hasta el destino: 1 días
Combustible para llegar: 8 unidades
Combustible para regresar: 8 unidades
Reserva de combustible: 10 unidades
Combustible total requerido: 26 unidades
Oxígeno requerido: 9 unidades
Energía requerida: 20 unidades
Provisiones requeridas: 5 unidades
PS C:\Users\xmlsc\OneDrive\Desktop\Principios de programacion #1\mision-orbita>
```

CP-03: Mision larga con más tripulantes

```

PS C:\Users\xmlsc\OneDrive\Desktop\Principios de programacion #1\mision-orbita
> & C:\Users\xmlsc\AppData\Local\Programs\Python\Python314\python.exe "c:/User
s/xmlsc/OneDrive/Desktop/Principios de programacion #1/mision-orbita/avance1/e
jercicio1_recursos.py"
Nombre de la misión: Titan
Cantidad de tripulantes: 4
Días estimados para llegar al destino: 10

Misión: Titan
Tripulantes: 4
Duración estimada hasta el destino: 10 días
Combustible para llegar: 80 unidades
Combustible para regresar: 80 unidades
Reserva de combustible: 10 unidades
Combustible total requerido: 170 unidades
Oxígeno requerido: 165 unidades
Energía requerida: 110 unidades
Provisiones requeridas: 83 unidades
PS C:\Users\xmlsc\OneDrive\Desktop\Principios de programacion #1\mision-orbita
>

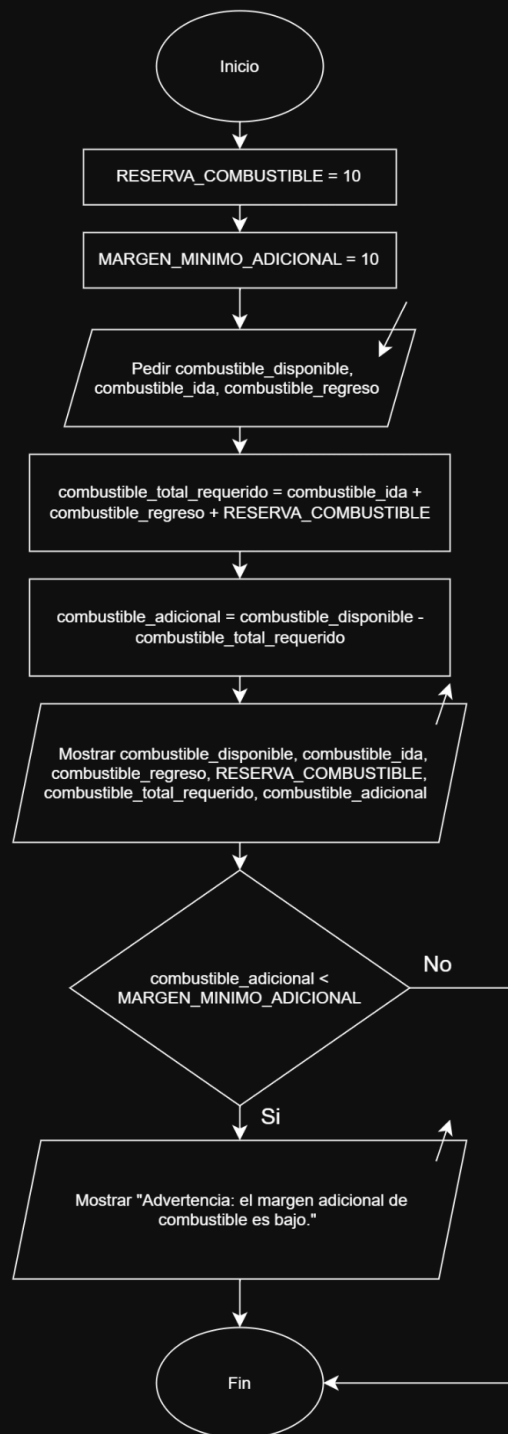
```

EJERCICIO 2: Combustible**Tabla de variables:**

Nombre	Tipo de dato	Categoría	Descripción	Valor inicial / restricciones
RESERVA_COMBUSTIBLE	int	Constante	Reserva de emergencia que toda misión debe llevar	10
MARGEN_MINIMO_ADICIONAL	int	Constante	Límite por debajo del cual se advierte margen bajo	10
combustible_disponible	int	Entrada	Combustible con que cuenta la nave	Entero positivo
combustible_ida	int	Entrada	Combustible requerido para llegar al destino	Entero positivo
combustible_regreso	int	Entrada	Combustible requerido para regresar	Entero positivo
combustible_total_requerido	int	Salida	Combustible total que exige la misión (ida + regreso + reserva)	combustible_ida + combustible_regreso + RESERVA_COMBUSTIBLE
combustible_adicional	int	Salida	Combustible que sobra después de cubrir la misión	combustible_disponible - combustible_total_requerido

Diagrama de flujo:

Ejercicio 2: Verificacion de combustible



Código en python:

```
# =====
# Archivo: ejercicio2_combustible.py
# Curso: SOFT-01 Principios de Programación 1 - Sección SCV3
# Integrantes: Julián Salazar Solis
# Fecha: 28/09/2026      Versión: 1.0
# Descripción: Verifica si una nave posee suficiente combustible
#               para la misión y advierte si el margen es bajo.
# =====

# -----
# Declaración de constantes
# -----

RESERVA_COMBUSTIBLE = 10
MARGEN_MINIMO_ADICIONAL = 10

# -----
# Entrada de datos
# -----
combustible_disponible = int(input("Combustible disponible (unidades):
"))
combustible_ida = int(input("Combustible requerido para llegar al
destino (unidades): "))
combustible_regreso = int(input("Combustible requerido para regresar
(unidades): "))

# -----
# Procesamiento
# La reserva se suma una sola vez porque cubre emergencias de toda la
misión.
# -----
combustible_total_requerido = combustible_ida + combustible_regreso +
RESERVA_COMBUSTIBLE
combustible_adicional = combustible_disponible -
combustible_total_requerido

# -----
# Salida de resultados
# Estos seis datos se muestran siempre, sin importar el margen.
# -----
print()
print("Combustible disponible:", combustible_disponible, "unidades")
```

```

print("Combustible requerido para llegar:", combustible_ida,
"unidades")
print("Combustible requerido para regresar:", combustible_regreso,
"unidades")
print("Reserva de emergencia:", RESERVA_COMBUSTIBLE, "unidades")
print("Combustible total requerido:", combustible_total_requerido,
"unidades")
print("Combustible adicional disponible:", combustible_adicional,
"unidades")

# -----
# Decisión
# Se usa < y no <= porque un margen de exactamente 10 unidades
# todavía se considera aceptable. Sin else: solo se advierte en este
# caso.
# -----
if combustible_adicional < MARGEN_MINIMO_ADICIONAL:
    print("Advertencia: el margen adicional de combustible es bajo.")

```

Casos de prueba:

ID	Ejercicio	Escenario	Datos de entrada	Resultado esperado	Resultado Obtenido	Estado
CP-0 4	Ejercicio 2	Combustible suficiente con buen margen	Disponible = 150 Ida = 40 Regreso = 40	Total = 90 Adicional = 60 Sin advertencia	Combustible disponible: 150 unidades Combustible requerido para llegar: 40 unidades Combustible requerido para regresar: 40 unidades Reserva de emergencia: 10 unidades Combustible total requerido: 90 unidades Combustible adicional disponible: 60 unidades	Aprobado
CP-0 5	Ejercicio 2	Límite: adicional exactamente igual al margen	Disponible = 100 Ida = 40 Regreso = 40	Total = 90 Adicional = 10 Sin advertencia (10 no es menor que 10)	Combustible disponible: 100 unidades Combustible requerido para llegar: 40 unidades Combustible requerido para regresar: 40 unidades Reserva de emergencia: 10 unidades Combustible total requerido: 90 unidades Combustible adicional disponible: 10 unidades	Aprobado
CP-0 6	Ejercicio 2	Poco margen adicional	Disponible = 95 Ida = 40 Regreso = 40	Total = 90 Adicional = 5 Muestra advertencia	Combustible disponible: 95 unidades Combustible requerido para llegar: 40 unidades Combustible requerido para regresar: 40 unidades Reserva de emergencia: 10 unidades Combustible total requerido: 90 unidades Combustible adicional disponible: 5 unidades	Aprobado

					Advertencia: el margen adicional de combustible es bajo.	
CP-07	Ejercicio 2	Recursos exactamente iguales al mínimo	Disponible = 90 Ida = 40 Regreso = 40	Total = 90 Adicional = 0 Muestra advertencia	Combustible disponible: 90 unidades Combustible requerido para llegar: 40 unidades Combustible requerido para regresar: 40 unidades Reserva de emergencia: 10 unidades Combustible total requerido: 90 unidades Combustible adicional disponible: 0 unidades Advertencia: el margen adicional de combustible es bajo.	Aprobado
CP-08	Ejercicio 2	Combustible suficiente solo para llegar	Disponible = 50 Ida = 40 Regreso = 40	Total = 90 Adicional = -40 Muestra advertencia	Combustible disponible: 50 unidades Combustible requerido para llegar: 40 unidades Combustible requerido para regresar: 40 unidades Reserva de emergencia: 10 unidades Combustible total requerido: 90 unidades Combustible adicional disponible: -40 unidades Advertencia: el margen adicional de combustible es bajo.	Aprobado

Evidencia de ejecución:

CP-04: Combustible suficiente con buen margen

```
PS C:\Users\xmlsc\OneDrive\Desktop\Principios de programacion #1\mision-orbita> & C:\Users\xmlsc\OneDrive\Desktop\Principios de programacion #1\mision-orbita\avance1\ejercicio2_combustible
Combustible disponible (unidades): 150
Combustible requerido para llegar al destino (unidades): 40
Combustible requerido para regresar (unidades): 40

Combustible disponible: 150 unidades
Combustible requerido para llegar: 40 unidades
Combustible requerido para regresar: 40 unidades
Reserva de emergencia: 10 unidades
Combustible total requerido: 90 unidades
Combustible adicional disponible: 60 unidades
PS C:\Users\xmlsc\OneDrive\Desktop\Principios de programacion #1\mision-orbita>
```

CP-05: Límite, adicional exactamente igual al margen

```
PS C:\Users\xmlsc\OneDrive\Desktop\Principios de programacion #1\mision-orbita> & C:\Users\xmlsc\AppData\Local\Microsoft\Windows\Apps\OneDrive\OneDrive\Desktop\Principios de programacion #1\mision-orbita\avance1\ejercicio2_combustible.py"
Combustible disponible (unidades): 100
Combustible requerido para llegar al destino (unidades): 40
Combustible requerido para regresar (unidades): 40

Combustible disponible: 100 unidades
Combustible requerido para llegar: 40 unidades
Combustible requerido para regresar: 40 unidades
Reserva de emergencia: 10 unidades
Combustible total requerido: 90 unidades
Combustible adicional disponible: 10 unidades
PS C:\Users\xmlsc\OneDrive\Desktop\Principios de programacion #1\mision-orbita>
```

CP-06: Poco margen adicional

```

PS C:\Users\xmlsc\OneDrive\Desktop\Principios de programacion #1\mision-orbita> & C:\Users\xmlsc\AppData\Local\Microsoft\Windows\Apps\OneDrive\OneDrive\Principios de programacion #1\mision-orbita\avance1\ejercicio2_combustible.py"
Combustible disponible (unidades): 95
Combustible requerido para llegar al destino (unidades): 40
Combustible requerido para regresar (unidades): 40

Combustible disponible: 95 unidades
Combustible requerido para llegar: 40 unidades
Combustible requerido para regresar: 40 unidades
Reserva de emergencia: 10 unidades
Combustible total requerido: 90 unidades
Combustible adicional disponible: 5 unidades
Advertencia: el margen adicional de combustible es bajo.
PS C:\Users\xmlsc\OneDrive\Desktop\Principios de programacion #1\mision-orbita>

```

CP-07: Recursos exactamente iguales al mínimo

```

PS C:\Users\xmlsc\OneDrive\Desktop\Principios de programacion #1\mision-orbita> & C:\Users\xmlsc\AppData\Local\Microsoft\Windows\Apps\OneDrive\OneDrive\Principios de programacion #1\mision-orbita\avance1\ejercicio2_combustible.py"
Combustible disponible (unidades): 90
Combustible requerido para llegar al destino (unidades): 40
Combustible requerido para regresar (unidades): 40

Combustible disponible: 90 unidades
Combustible requerido para llegar: 40 unidades
Combustible requerido para regresar: 40 unidades
Reserva de emergencia: 10 unidades
Combustible total requerido: 90 unidades
Combustible adicional disponible: 0 unidades
Advertencia: el margen adicional de combustible es bajo.
PS C:\Users\xmlsc\OneDrive\Desktop\Principios de programacion #1\mision-orbita>

```

CP-08: Combustible suficiente solo para llegar

```

PS C:\Users\xmlsc\OneDrive\Desktop\Principios de programacion #1\mision-orbita> & C:\Users\xmlsc\AppData\Local\Microsoft\Windows\Apps\OneDrive\OneDrive\Principios de programacion #1\mision-orbita\avance1\ejercicio2_combustible.py"
Combustible disponible (unidades): 50
Combustible requerido para llegar al destino (unidades): 40
Combustible requerido para regresar (unidades): 40

Combustible disponible: 50 unidades
Combustible requerido para llegar: 40 unidades
Combustible requerido para regresar: 40 unidades
Reserva de emergencia: 10 unidades
Combustible total requerido: 90 unidades
Combustible adicional disponible: -40 unidades
Advertencia: el margen adicional de combustible es bajo.
PS C:\Users\xmlsc\OneDrive\Desktop\Principios de programacion #1\mision-orbita>

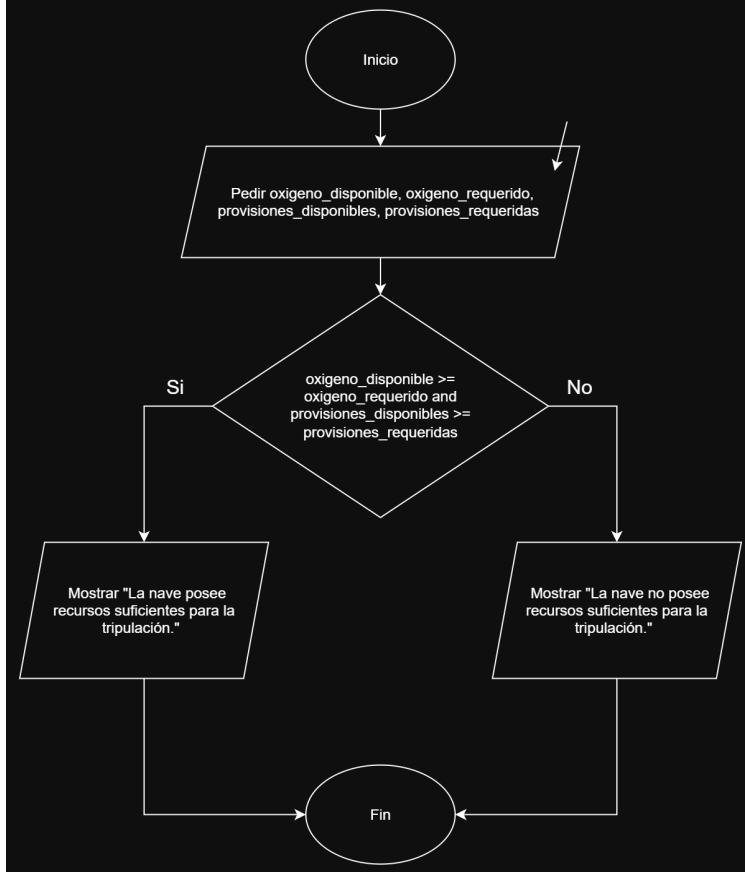
```

EJERCICIO 3: Soporte a la tripulación**Tabla de variables:**

Nombre	Tipo de dato	Categoría	Descripción	Valor inicial / restricciones
oxigeno_disponible	int	Entrada	Oxígeno con que cuenta la nave	Entero positivo
oxigeno_requerido	int	Entrada	Oxígeno que necesita la tripulación durante el viaje	Entero positivo
provisiones_disponibles	int	Entrada	Provisiones con que cuenta la nave	Entero positivo
provisiones_requeridas	int	Entrada	Provisiones que necesita la tripulación durante el viaje	Entero positivo

Diagrama de flujo:

Ejercicio 3: Verificación del soporte de tripulación



Código en python:

```

# =====
# Archivo: ejercicio3_soporte_tripulacion.py
# Curso: SOFT-01 Principios de Programación 1 - Sección SCV3
# Integrantes: Julián Salazar Solis
# Fecha: 29/09/2026 Versión: 1.0
# Descripción: Verifica si la nave posee oxígeno y provisiones
#              suficientes para mantener a la tripulación.
# =====

# -----
# Entrada de datos
# -----

oxigeno_disponible = int(input("Oxígeno disponible (unidades): "))
oxigeno_requerido = int(input("Oxígeno requerido (unidades): "))

```

```

provisiones_disponibles = int(input("Provisiones disponibles
(unidades): "))
provisiones_requeridas = int(input("Provisiones requeridas (unidades):
"))

# -----
# Decisión y salida de resultados
# Se usa and porque la tripulación solo tiene soporte adecuado si
# ambos recursos alcanzan; con >= se acepta el caso exacto del mínimo.
# Todavía no se indica cuál recurso falla, según la consigna.
# -----

print()
if oxigeno_disponible >= oxigeno_requerido and provisiones_disponibles
>= provisiones_requeridas:
    print("La nave posee recursos suficientes para la tripulación.")
else:
    print("La nave no posee recursos suficientes para la tripulación.")

```

Casos de prueba:

ID	Ejercicio	Escenario	Datos de entrada	Resultado esperado	Resultado Obtenido	Estado
CP-09	Ejercicio 3	Ambos recursos suficientes	Oxígeno disponible = 50 Oxígeno requerido = 35 Provisiones disponibles = 30 Provisiones requeridas = 18	"La nave posee recursos suficientes para la tripulación."	La nave posee recursos suficientes para la tripulación.	Aprobado
CP-10	Ejercicio 3	Límite: ambos exactamente iguales al mínimo	Oxígeno disponible = 35 Oxígeno requerido = 35 Provisiones disponibles = 18 Provisiones requeridas = 18	"La nave posee recursos suficientes para la tripulación."	La nave posee recursos suficientes para la tripulación.	Aprobado
CP-11	Ejercicio 3	Oxígeno insuficiente, provisiones suficientes	Oxígeno disponible = 34 Oxígeno requerido = 35 Provisiones disponibles = 20 Provisiones requeridas = 18	"La nave no posee recursos suficientes para la tripulación."	La nave no posee recursos suficientes para la tripulación.	Aprobado
CP-12	Ejercicio 3	Provisiones insuficientes, oxígeno suficiente	Oxígeno disponible = 40 Oxígeno requerido = 35 Provisiones disponibles = 17 Provisiones requeridas = 18	"La nave no posee recursos suficientes para la tripulación."	La nave no posee recursos suficientes para la tripulación.	Aprobado
CP-13	Ejercicio 3	Ambos recursos insuficientes	Oxígeno disponible = 10 Oxígeno requerido = 35 Provisiones disponibles = 5 Provisiones requeridas = 18	"La nave no posee recursos suficientes para la tripulación."	La nave no posee recursos suficientes para la tripulación.	Aprobado

Evidencia de ejecución:

CP-09: Ambos recursos suficientes

```
PS C:\Users\xmlsc\> & C:\Users\xmlsc\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/xmlsc/OneDrive/Desktop/Principios de programacion #1/mision-orbita/avance1/ejercicio3_soporte_tripulacion.py"
Oxígeno disponible (unidades): 50
Oxígeno requerido (unidades): 35
Provisiones disponibles (unidades): 30
Provisiones requeridas (unidades): 18

La nave posee recursos suficientes para la tripulación.
PS C:\Users\xmlsc\OneDrive\Desktop\Principios de programacion #1\mision-orbita> █
```

CP-10: Límite, ambos exactamente iguales al mínimo

```
PS C:\Users\xmlsc\OneDrive\Desktop\Principios de programacion #1\mision-orbita> & C:\Users\xmlsc\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/xmlsc/OneDrive/Desktop/Principios de programacion #1/mision-orbita/avance1/ejercicio3_soporte_tripulacion.py"
Oxígeno disponible (unidades): 35
Oxígeno requerido (unidades): 35
Provisiones disponibles (unidades): 18
Provisiones requeridas (unidades): 18

La nave posee recursos suficientes para la tripulación.
PS C:\Users\xmlsc\OneDrive\Desktop\Principios de programacion #1\mision-orbita> █
```

CP-11: Oxígeno insuficiente, provisiones suficientes

```
PS C:\Users\xmlsc\OneDrive\Desktop\Principios de programacion #1\mision-orbita> & C:\Users\xmlsc\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/xmlsc/OneDrive/Desktop/Principios de programacion #1/mision-orbita/avance1/ejercicio3_soporte_tripulacion.py"
Oxígeno disponible (unidades): 34
Oxígeno requerido (unidades): 35
Provisiones disponibles (unidades): 20
Provisiones requeridas (unidades): 18

La nave no posee recursos suficientes para la tripulación.
PS C:\Users\xmlsc\OneDrive\Desktop\Principios de programacion #1\mision-orbita> █
```

CP-12: Provisiones insuficientes, oxígeno suficiente

```
PS C:\Users\xmlsc\OneDrive\Desktop\Principios de programacion #1\mision-orbita> & C:\Users\xmlsc\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/xmlsc/OneDrive/Desktop/Principios de programacion #1/mision-orbita/avance1/ejercicio3_soporte_tripulacion.py"
Oxígeno disponible (unidades): 40
Oxígeno requerido (unidades): 35
Provisiones disponibles (unidades): 17
Provisiones requeridas (unidades): 18

La nave no posee recursos suficientes para la tripulación.
PS C:\Users\xmlsc\OneDrive\Desktop\Principios de programacion #1\mision-orbita> █
```

CP-13: Ambos recursos insuficientes

```

PS C:\Users\xmlsc\OneDrive\Desktop\Principios de programacion #1\mision-orbita> & C:\Users\xmlsc\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/xmlsc/OneDrive/Desktop/Principios de programacion #1/mision-orbita/avance1/ejercicio3_soporte_tripulacion.py"
Oxígeno disponible (unidades): 10
Oxígeno requerido (unidades): 35
Provisiones disponibles (unidades): 5
Provisiones requeridas (unidades): 18

La nave no posee recursos suficientes para la tripulación.
PS C:\Users\xmlsc\OneDrive\Desktop\Principios de programacion #1\mision-orbita>

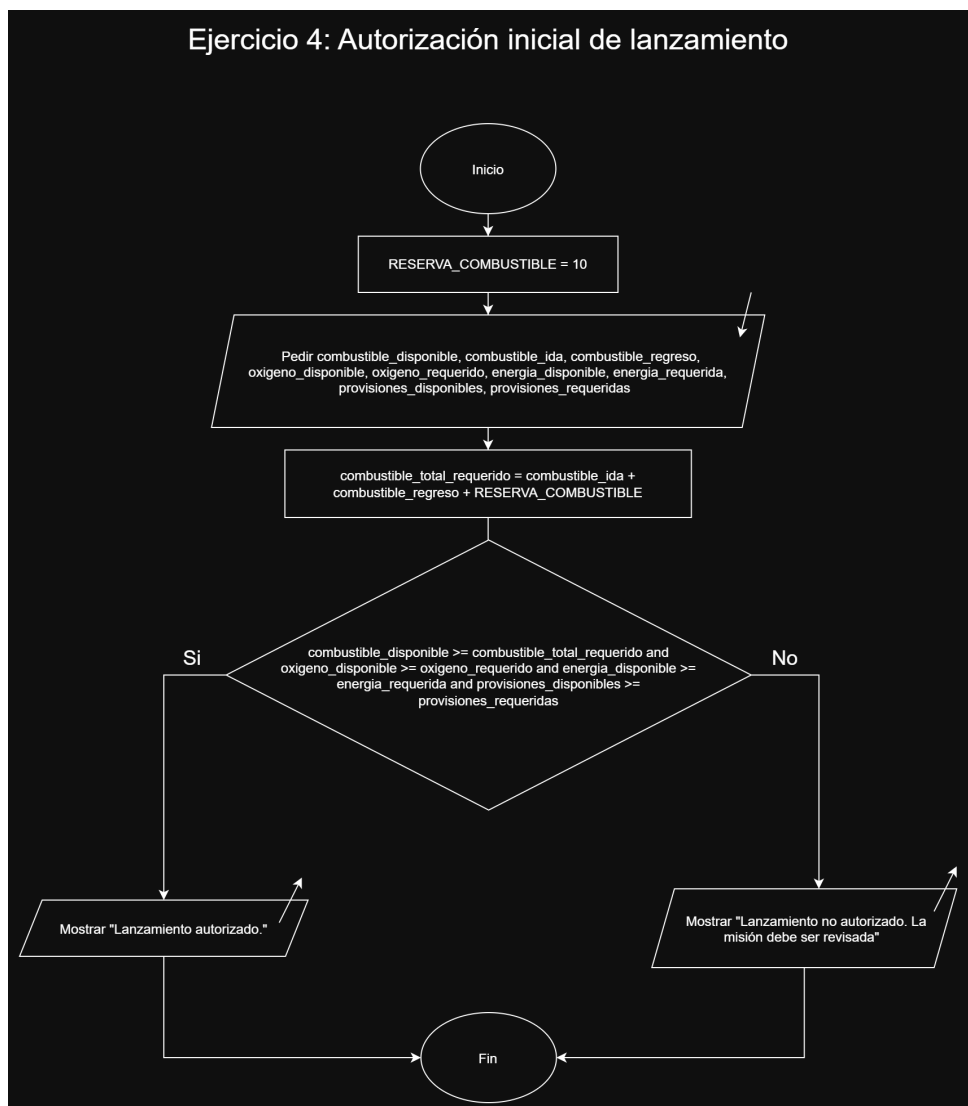
```

EJERCICIO 4: Autorización de lanzamiento**Tabla de variables:**

Nombre	Tipo de dato	Categoría	Descripción	Valor inicial / restricciones
RESERVA_COMBUSTIBLE	int	Constante	Reserva de emergencia que toda misión debe llevar	10
combustible_disponible	int	Entrada	Combustible con que cuenta la nave	Entero positivo
combustible_ida	int	Entrada	Combustible requerido para llegar al destino	Entero positivo
combustible_regreso	int	Entrada	Combustible requerido para regresar	Entero positivo
oxigeno_disponible	int	Entrada	Oxígeno con que cuenta la nave	Entero positivo
oxigeno_requerido	int	Entrada	Oxígeno que necesita la misión	Entero positivo
energia_disponible	int	Entrada	Energía con que cuenta la nave	Entero positivo
energia_requerida	int	Entrada	Energía que necesita la	Entero positivo

			misión	
provisiones_disponibles	int	Entrada	Provisiones con que cuenta la nave	Entero positivo
provisiones_requeridas	int	Entrada	Provisiones que necesita la misión	Entero positivo
combustible_total_requerido	int	Intermedia	Combustible que exige la misión con ida, regreso y reserva	combustible_ida + combustible_regreso + RESERVA_COMBUSTIBLE

Diagrama de flujo:



Código en python:

```
# =====
# Archivo: ejercicio4_autorizacion.py
# Curso: SOFT-01 Principios de Programación 1 - Sección SCV3
# Integrantes: Julián Salazar Solís
# Fecha: 30/09/2026      Versión: 1.0
# Descripción: Verifica si una misión puede ser autorizada para
#               el lanzamiento según sus recursos disponibles.
# =====

# -----
# Declaración de constantes
# -----
RESERVA_COMBUSTIBLE = 10

# -----
# Entrada de datos
# -----
combustible_disponible = int(input("Combustible disponible (unidades):
"))
combustible_ida = int(input("Combustible requerido para llegar al
destino (unidades): "))
combustible_regreso = int(input("Combustible requerido para regresar
(unidades): "))
oxigeno_disponible = int(input("Oxígeno disponible (unidades): "))
oxigeno_requerido = int(input("Oxígeno requerido (unidades): "))
energia_disponible = int(input("Energía disponible (unidades): "))
energia_requerida = int(input("Energía requerida (unidades): "))
provisiones_disponibles = int(input("Provisiones disponibles
(unidades): "))
provisiones_requeridas = int(input("Provisiones requeridas (unidades):
"))

# -----
# Procesamiento
# El combustible total incluye el regreso y la reserva, para no
# autorizar misiones que solo pueden llegar al destino.
# -----
combustible_total_requerido = combustible_ida + combustible_regreso +
RESERVA_COMBUSTIBLE

# -----
# Decisión y salida de resultados
```

```
# Se usa and porque el lanzamiento solo se autoriza si los cuatro
# recursos alcanzan; >= acepta el caso exacto del mínimo.
# Todavía no se indica cuál recurso provocó el rechazo.
# -----
print()
if (combustible_disponible >= combustible_total_requerido
    and oxigeno_disponible >= oxigeno_requerido
    and energia_disponible >= energia_requerida
    and provisiones_disponibles >= provisiones_requeridas):
    print("Lanzamiento autorizado.")
else:
    print("Lanzamiento no autorizado. La misión debe ser revisada.")
```

Casos de prueba:

ID	Ejercicio	Escenario	Datos de entrada (en el orden en que los pide el programa)	Resultado esperado	Resultado Obtenido	Estado
CP-14	Ejercicio 4	Todos los recursos suficientes	Combustible: 120, 40, 40 Oxígeno: 80 / 65 Energía: 70 / 60 Provisiones: 40 / 33	"Lanzamiento autorizado."	Lanzamiento autorizado.	Aprobado
CP-15	Ejercicio 4	Límite: todos exactamente iguales al mínimo	Combustible: 90, 40, 40 Oxígeno: 65 / 65 Energía: 60 / 60 Provisiones: 33 / 33	"Lanzamiento autorizado."	Lanzamiento autorizado.	Aprobado
CP-16	Ejercicio 4	Combustible suficiente solo para llegar	Combustible: 50, 40, 40 Oxígeno: 80 / 65 Energía: 70 / 60 Provisiones: 40 / 33	"Lanzamiento no autorizado. La misión debe ser revisada."	Lanzamiento no autorizado. La misión debe ser revisada.	Aprobado
CP-17	Ejercicio 4	Combustible cubre ida y regreso pero no la reserva	Combustible: 89, 40, 40 Oxígeno: 80 / 65 Energía: 70 / 60 Provisiones: 40 / 33	"Lanzamiento no autorizado. La misión debe ser revisada."	Lanzamiento no autorizado. La misión debe ser revisada.	Aprobado
CP-18	Ejercicio 4	Oxígeno insuficiente	Combustible: 120, 40, 40 Oxígeno: 64 / 65 Energía: 70 / 60 Provisiones: 40 / 33	"Lanzamiento no autorizado. La misión debe ser revisada."	Lanzamiento no autorizado. La misión debe ser revisada.	Aprobado
CP-19	Ejercicio 4	Energía insuficiente	Combustible: 120, 40, 40 Oxígeno: 80 / 65 Energía: 59 / 60 Provisiones: 40 / 33	"Lanzamiento no autorizado. La misión debe ser revisada."	Lanzamiento no autorizado. La misión debe ser revisada.	Aprobado
CP-20	Ejercicio 4	Provisiones insuficientes	Combustible: 120, 40, 40 Oxígeno: 80 / 65 Energía: 70 / 60 Provisiones: 32 / 33	"Lanzamiento no autorizado. La misión debe ser revisada."	Lanzamiento no autorizado. La misión debe ser revisada.	Aprobado

Evidencia de ejecución:

CP-14: Todos los recursos suficientes

```
PS C:\Users\xmlsc\OneDrive\Desktop\Principios de programacion #1\mision-orbita
> & C:\Users\xmlsc\AppData\Local\Programs\Python\Python314\python.exe "c:/User
s/xmlsc/OneDrive/Desktop/Principios de programacion #1/mision-orbita/avance1/e
jercicio4_autorizacion.py"
Combustible disponible (unidades): 120
Combustible requerido para llegar al destino (unidades): 40
Combustible requerido para regresar (unidades): 40
Oxígeno disponible (unidades): 80
Oxígeno requerido (unidades): 65
Energía disponible (unidades): 70
Energía requerida (unidades): 60
Provisiones disponibles (unidades): 40
Provisiones requeridas (unidades): 33

Lanzamiento autorizado.
PS C:\Users\xmlsc\OneDrive\Desktop\Principios de programacion #1\mision-orbita
>
```

CP-15: Límite, todos exactamente iguales al mínimo

```
PS C:\Users\xmlsc\OneDrive\Desktop\Principios de programacion #1\mision-orbita
> & C:\Users\xmlsc\AppData\Local\Programs\Python\Python314\python.exe "c:/User
s/xmlsc/OneDrive/Desktop/Principios de programacion #1/mision-orbita/avance1/e
jercicio4_autorizacion.py"
Combustible disponible (unidades): 90
Combustible requerido para llegar al destino (unidades): 40
Combustible requerido para regresar (unidades): 40
Oxígeno disponible (unidades): 65
Oxígeno requerido (unidades): 65
Energía disponible (unidades): 60
Energía requerida (unidades): 60
Provisiones disponibles (unidades): 33
Provisiones requeridas (unidades): 33

Lanzamiento autorizado.
PS C:\Users\xmlsc\OneDrive\Desktop\Principios de programacion #1\mision-orbita
>
```

CP-16: Combustible suficiente solo para llegar

```
PS C:\Users\xmlsc\OneDrive\Desktop\Principios de programacion #1\mision-orbita
> & C:\Users\xmlsc\AppData\Local\Programs\Python\Python314\python.exe "c:/User
s/xmlsc/OneDrive/Desktop/Principios de programacion #1/mision-orbita/avance1/e
jercicio4_autorizacion.py"
Combustible disponible (unidades): 50
Combustible requerido para llegar al destino (unidades): 40
Combustible requerido para regresar (unidades): 40
Oxígeno disponible (unidades): 80
Oxígeno requerido (unidades): 65
Energía disponible (unidades): 70
Energía requerida (unidades): 60
Provisiones disponibles (unidades): 40
Provisiones requeridas (unidades): 33

Lanzamiento no autorizado. La misión debe ser revisada.
PS C:\Users\xmlsc\OneDrive\Desktop\Principios de programacion #1\mision-orbita
>
```

CP-17: Combustible cubre ida y regreso pero no la reserva

```
PS C:\Users\xmlsc\OneDrive\Desktop\Principios de programacion #1\mision-orbita
> & C:\Users\xmlsc\AppData\Local\Programs\Python\Python314\python.exe "c:/User
s/xmlsc/OneDrive/Desktop/Principios de programacion #1/mision-orbita/avance1/e
jercicio4_autorizacion.py"
Combustible disponible (unidades): 89
Combustible requerido para llegar al destino (unidades): 40
Combustible requerido para regresar (unidades): 40
Oxígeno disponible (unidades): 80
Oxígeno requerido (unidades): 65
Energía disponible (unidades): 70
Energía requerida (unidades): 60
Provisiones disponibles (unidades): 40
Provisiones requeridas (unidades): 33

Lanzamiento no autorizado. La misión debe ser revisada.
PS C:\Users\xmlsc\OneDrive\Desktop\Principios de programacion #1\mision-orbita
>
```

CP-18: Oxígeno insuficiente

```
PS C:\Users\xmlsc\OneDrive\Desktop\Principios de programacion #1\mision-orbita
> & C:\Users\xmlsc\AppData\Local\Programs\Python\Python314\python.exe "c:/User
s/xmlsc/OneDrive/Desktop/Principios de programacion #1/mision-orbita/avance1/e
jercicio4_autorizacion.py"
Combustible disponible (unidades): 120
Combustible requerido para llegar al destino (unidades): 40
Combustible requerido para regresar (unidades): 40
Oxígeno disponible (unidades): 64
Oxígeno requerido (unidades): 65
Energía disponible (unidades): 70
Energía requerida (unidades): 60
Provisiones disponibles (unidades): 40
Provisiones requeridas (unidades): 33

Lanzamiento no autorizado. La misión debe ser revisada.
PS C:\Users\xmlsc\OneDrive\Desktop\Principios de programacion #1\mision-orbita
>
```

CP-19: Energía insuficiente

```
PS C:\Users\xmlsc\OneDrive\Desktop\Principios de programacion #1\mision-orbita
> & C:\Users\xmlsc\AppData\Local\Programs\Python\Python314\python.exe "c:/User
s/xmlsc/OneDrive/Desktop/Principios de programacion #1/mision-orbita/avance1/e
jercicio4_autorizacion.py"
Combustible disponible (unidades): 120
Combustible requerido para llegar al destino (unidades): 40
Combustible requerido para regresar (unidades): 40
Oxígeno disponible (unidades): 80
Oxígeno requerido (unidades): 65
Energía disponible (unidades): 59
Energía requerida (unidades): 60
Provisiones disponibles (unidades): 40
Provisiones requeridas (unidades): 33

Lanzamiento no autorizado. La misión debe ser revisada.
PS C:\Users\xmlsc\OneDrive\Desktop\Principios de programacion #1\mision-orbita
>
```

CP-20: Provisiones insuficientes

```
PS C:\Users\xmlsc\OneDrive\Desktop\Principios de programacion #1\mision-orbita
> & C:\Users\xmlsc\AppData\Local\Programs\Python\Python314\python.exe "c:/User
s/xmlsc/OneDrive/Desktop/Principios de programacion #1/mision-orbita/avance1/e
jercicio4_autorizacion.py"
Combustible disponible (unidades): 120
Combustible requerido para llegar al destino (unidades): 40
Combustible requerido para regresar (unidades): 40
Oxígeno disponible (unidades): 80
Oxígeno requerido (unidades): 65
Energía disponible (unidades): 70
Energía requerida (unidades): 60
Provisiones disponibles (unidades): 32
Provisiones requeridas (unidades): 33

Lanzamiento no autorizado. La misión debe ser revisada.
PS C:\Users\xmlsc\OneDrive\Desktop\Principios de programacion #1\mision-orbita
> █
```